INTRODUCTION TO MACHINE LEARNING AND DATA MINING RESEARCH PROJECT

Group 7

Inês Neves 202206252
Beatriz Moreira 202208293

U.PORTO
FEUP FACULDADE DE ENGENHARIA UNIVERSIDADE DO PORTO

Selected Algorithm: XGBoost (Extreme Gradient Boosting)

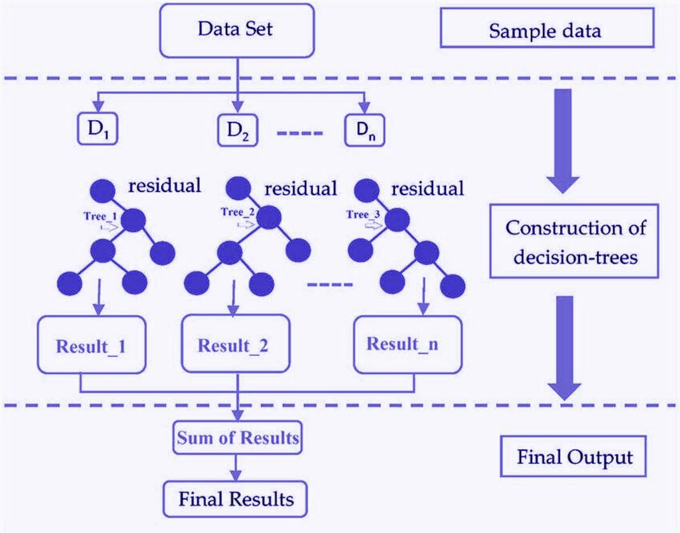
Description:

→ Ensemble learning algorithm based on gradient boosting of decision trees
→ Builds trees sequentially, learning from the residual errors of previous trees

Key Steps:

→ Initialize predictions
→ Compute gradients and hessians
→ Build a regression tree
→ Compute leaf weights
→ Update predictions
→ Repeat for each boosting round
→ Make final predictions

XGBoost Model Diagram



Source: General architecture of XGBoost (ResearchGate)

Selected Data Characteristic: Noise/Outliers

Impact of Noise/Outliers on XGBoost

→ Noise **reduces model stability:** training less robust and predictions more variable

→ Outliers produce **large residuals (gradients)**, disproportionately **affecting tree updates**

→ Large gradients from noisy points can **dominate split decisions**, creating overly specific splits

→ Extreme values can **skew leaf weight calculations**, leading to **highly biased predictions** in small regions → **OVERFITTING**

Outlier Detection (Implementation Code)

→ **IQR Method -** for numerical features.

→ For each dataset:

- Calculate Q1, Q3 → IQR = Q3 - Q1
- **Values outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR]** are flagged as **outliers**
- Compute **outlier percentage per dataset**

Proposal: Gradient clipping at per-sample level

Motivation

→ **Goal:** Reduce influence of outliers to stabilize training and improve robustness

→ **Why Gradient Clipping:**

- Limits the magnitude of each sample's gradient
- Prevents extreme gradients from disproportionately influencing split decisions and leaf values

→ **Expected Effect:**
More robust trees that generalize better in noisy or outlier-heavy datasets

Description

→ **How it works on XGBoost:**

- Each training sample contributes a gradient to both:
  → Leaf value calculation
  → Split gain computation
- Clip the gradient to a predefined threshold if it exceeds the limit
- Reduces the influence of outlying points on tree construction

→ **Source:** *Qian et al. (2021)* on incremental gradient methods

Empirical Study: Experimental Setup

Datasets and their Characteristics

→ **Source:** OpenML-CC18 Curated Classification benchmark

→ **Dataset selection:** Binary Classification Datasets

→ **Data Characteristic:** Noise/Outliers Percentage

→ **Other Characteristics analyzed per dataset:**

- Number of samples and features;
- Missing Values Percentage;
- Class Imbalance Ratio;
- High Cardinality Qualitative Attributes;
- Class Overlap

Hyperparameters

- **Original:**
  → Learning rate = 0.1
  → Max depth = 5
  → Subsample = 1.0
  → Regularization = 1.0
  → Gamma = 0.0

- **Gradient Clipping Variant:**
  Same hyperparameters as the original, for fair comparison, plus Gradient Clip = 0.5

Empirical Study: Experimental Setup

Performance Estimation Methodology

→ **Model Training:** Original XGBoost vs. Gradient-Clipped Variant

→ **Cross-validation:** Stratified **5-fold CV** to preserve class proportions, with each fold used once as test set

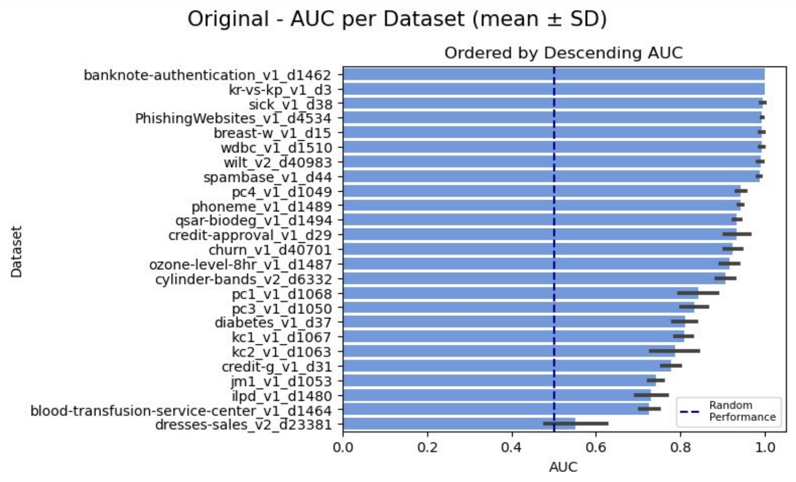
→ **Metrics Evaluated:**

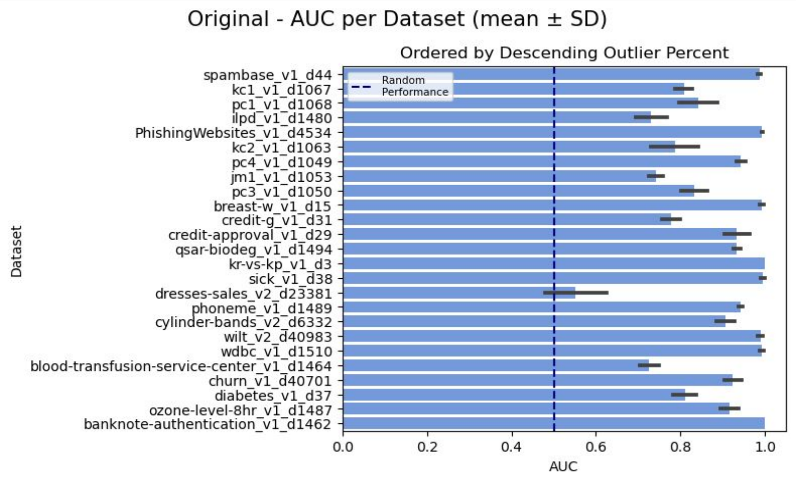
- Accuracy
- Precision
- Recall
- F1-Score
- AUC

→ **Metrics Aggregation:**

- Compute **metrics per fold;**
- **Average across folds** to get mean performance per dataset;
- Compare **variance across folds** to assess stability and robustness.

Experimental Results: Original XGBoost

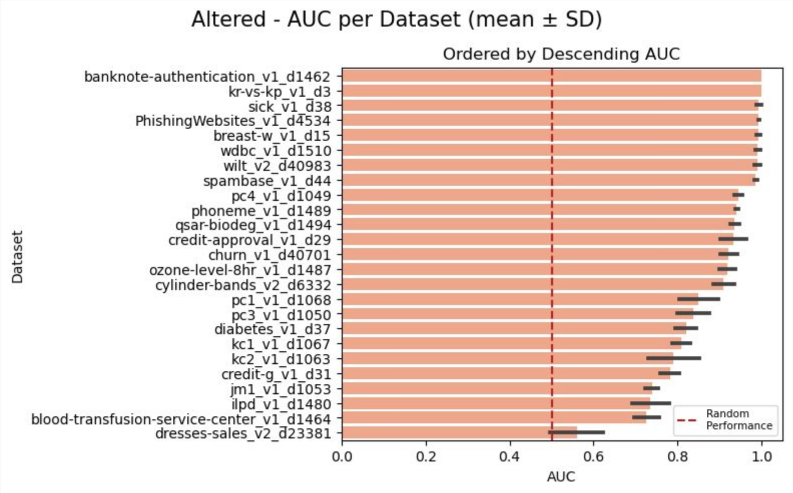


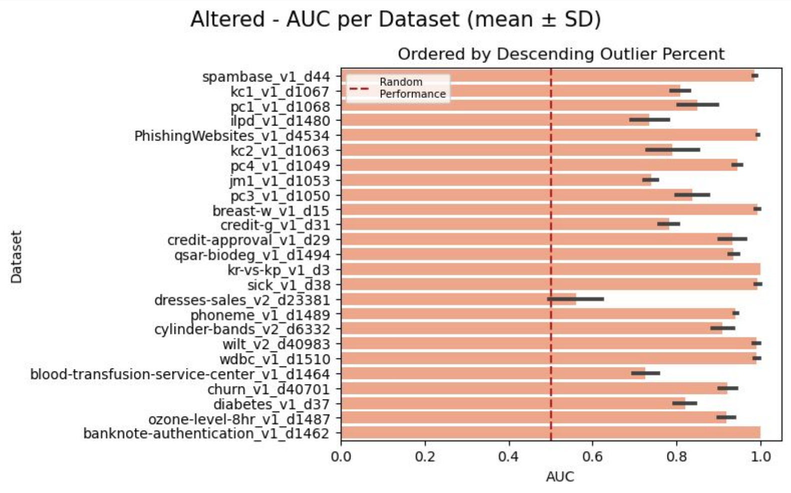


Analysis:

→ AUC was selected for comparison (other metrics are discussed in the notebook).
→ Most datasets show strong performance, with AUC values well above 0.5 and often around 0.8 or higher.
→ Ordering the results by outlier percentage shows no clear trend, meaning model performance does not appear to be affected by the proportion of outliers.

Experimental Results: Gradient Clipping Variant



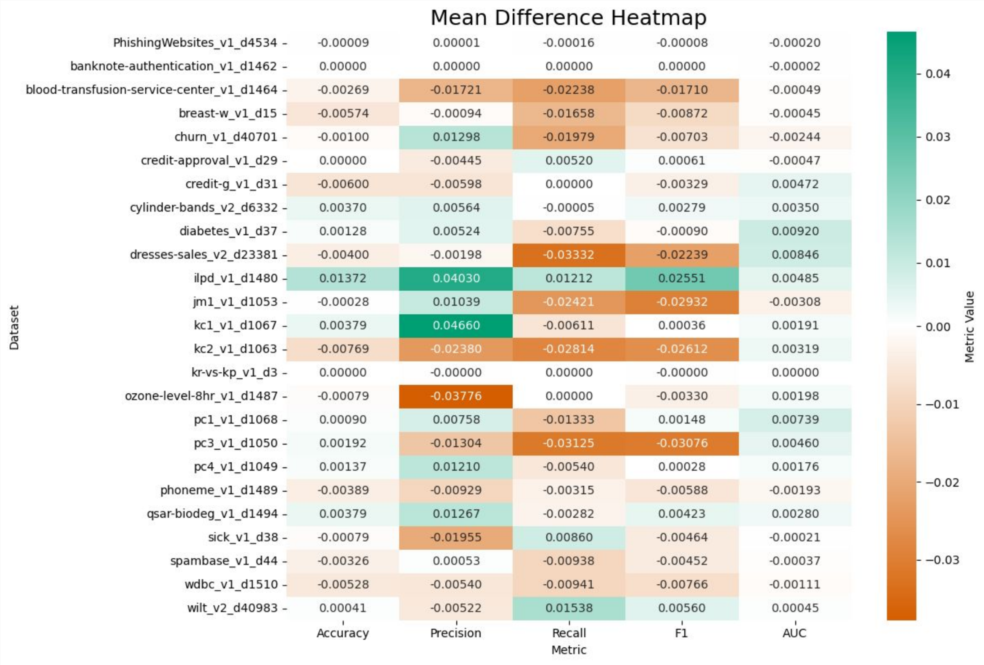


Analysis:

→ The Gradient Clipping version also performs well, with AUC values clearly above 0.5 and often between 0.85 and 0.95.

→ When results are ordered by outlier percentage, there is no clear pattern, showing that the model's AUC does not consistently change with more or fewer outliers.

Comparison: Original vs. Gradient Clipped Model



Analysis:

→ **Overall Minor Impact:** Majority of metric changes across all datasets are close to zero (±0.01 or smaller).

→ **Accuracy and AUC**: Negligible differences: the model's overall correctness and discriminatory power remained unchanged → **white/very light colors**.

→ **Precision and F1:** Minor fluctuations with a predominant trend of non-improvement → **light orange/brown hues**.

→ **Recall:** model sensitivity was slightly compromised in specific cases → **deeper orange.**

Additional Research: Synthetic Datasets

→ **Synthetic datasets:**

- Fixed number of samples and features;
- Randomly generated target.

→ **Outliers:** introduced at 20%, 30% and 40% of the feature values to simulate increasingly challenging conditions.

→ **Goal:** test how the algorithm and its altered version respond to extreme feature values.

Original Model Metrics

Dataset	Accuracy	Precision	Recall	F1	AUC
df_20	0.501	0.501	0.504	0.502	0.505
df_30	0.498	0.486	0.466	0.475	0.488
df_40	0.516	0.527	0.551	0.538	0.512

Altered Model Metrics

Dataset	Accuracy	Precision	Recall	F1	AUC
df_20	0.500	0.500	0.490	0.495	0.511
df_30	0.497	0.483	0.444	0.462	0.492
df_40	0.504	0.516	0.532	0.524	0.504

→ **Performance metrics** for both original and altered models ≈ **0.5**, reflecting random data.

→ **Minimal differences** between models, with no consistent improvement.

→ The alteration has **negligible impact** on performance under fully randomized conditions, confirming expected baseline behavior.

Conclusions

→ The alteration does not meaningfully change the model's predictive behaviour.

→ Results from real-world datasets and synthetic tests show the altered model performs essentially the same as the original.

→ No measurable benefit or harm was observed under the tested conditions.

→ The baseline algorithm maintains stable performance across all experiments.

Future Work

→ Test alteration on larger, more complex datasets.

→ Explore further hyperparameter tuning.

→ Investigate different outlier types.

→ Evaluate on structured datasets with clear patterns.

→ Assess robustness under varied data distributions and noise.